Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

OptiPack AI – AI-Assisted Omnichannel Order Fulfillment and Packaging Optimization (FA26SE036)

– Ho Chi Minh City, October 2026 –

Table of Contents

I. Record of Changes 3

II. Software Design Document 4

1. System Design 4

1.1 System Architecture 4

1.2 Package Diagram 5

1.3 Technology Declaration 6

2. Database Design 8

3. Main Business Workflows 10

3.1 End-to-End Overview 11

3.2 WF-01 Order Intake & Consolidation 12

3.3 WF-02 Picking 13

3.4 WF-03 Packing 14

3.5 WF-04 Shipping (GHN and Express Self-Delivery) 15

3.6 WF-05 Return / Exchange via AURELLE 17

3.7 WF-06 Stock Receiving by Lot 18

3.8 State Machine Diagrams 19

4. Detailed Design 20

4.1 Authentication and Account Management 20

4.2 Marketplace Integration and Order Intake 21

4.3 Warehouse and Inventory 21

4.4 Picking 21

4.5 Packing 22

4.6 Shipping 22

4.7 Returns and Exchanges 22

4.8 Marketplace Write-back 22

4.9 AURELLE Website 23

4.10 History and Notifications 23

I. Record of Changes

Date	A* M, D	In charge	Change Description
24/10/2026	A	ThuanNPM	Initial SDD for Review 2: system architecture, technology declaration, package descriptions, database design, main business workflows with activity diagrams, detailed design per feature.
25/10/2026	M	ThuanNPM	Aligned with the code: two web front ends plus the AURELLE Open API simulator; mobile app listed as planned (Iteration 3); Tiki / TikTok adapters not registered; CI jobs, release trigger, branches and Git hooks as configured in the repository; staging environment marked as not available; packing-plan states include approved; collection names corrected (customers, packaging_bags).
08/11/2026	M	ThuanNPM	Redrawn architecture diagram (Figure 1, HTTPS links to the packing optimizer and the AI service), ERD (Figure 3), overview and WF-01 to WF-06 activity diagrams (Figures 4–11) replaced by the latest draw.io exports; replaced the old Order Group state machine by the Order state machine (Figure 12); collection count 60; Tiki adapter status; test count; Diagrams row; packing-plan and shipment states completed in the workflow table (rejected / failed / superseded, cancelled); workflow owners filled.

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

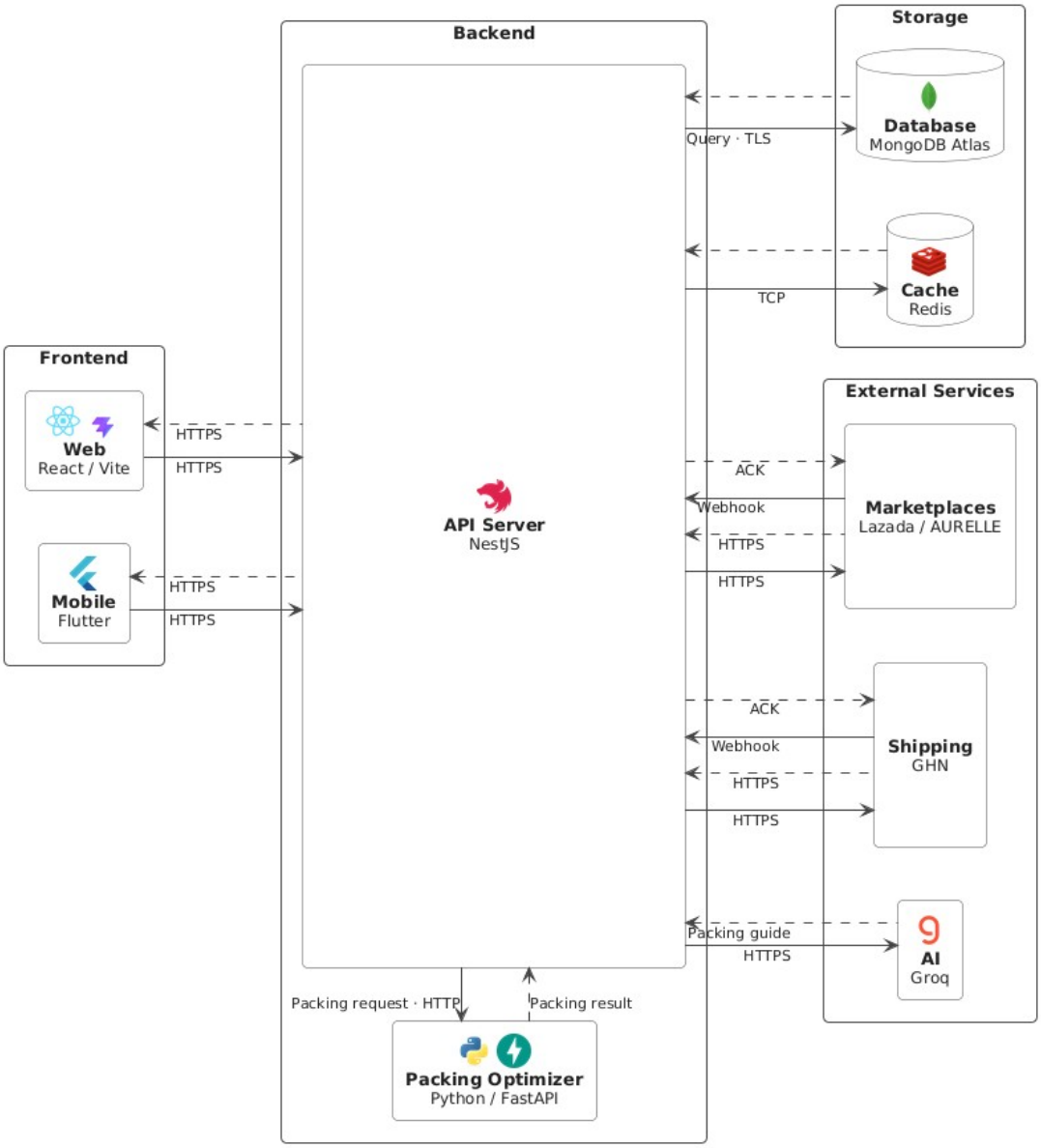


Figure 1. OptiPack AI system architecture

OptiPack AI follows a client–server architecture with a modular monolithic back end. Two web front ends (the OptiPack staff portal and the AURELLE website with its admin and developer pages) call one REST API; the AURELLE Open API used as the second marketplace runs as a separate simulator process (be/scripts/aurelle-mock-server.ts, port 4000) that reads the AURELLE data. Time-based work (order sync, packing-plan computation, status write-back) runs as scheduled jobs inside the back end. The 3D packing optimizer (Python / FastAPI) runs as a separate internal service called over HTTP and is optional.

The mobile app (Flutter) drawn in Figure 1 is planned for Iteration 3 and is not implemented in this release.

Component	Technology	Responsibility
OptiPack staff portal (fe)	React 19, Vite, Tailwind CSS, three.js	Web UI for the five staff roles; role-based menus and routes; 3D packing viewer.
AURELLE website (storefront)	Next.js 15, React 19	Customer shop (catalog, cart, checkout, order tracking, returns), web admin, developer portal.
Back end API (be)	NestJS 11 (Node.js, TypeScript)	REST API, authentication / RBAC, business logic of all modules, scheduled jobs, webhooks, AURELLE website back end and Open API mapper.
Packer service (packer)	Python 3.12, FastAPI, OR-Tools CP-SAT	Internal HTTP service that proves / improves 3D packing plans; if it is down the back end uses its built-in BRKGA solver.
Database	MongoDB Atlas (Mongoose ODM)	All business data (60 collections); multi-document transactions for stock and status changes.
Cache	Redis	Authentication state cache.
Mobile app	Flutter (planned)	Planned for Iteration 3; not implemented in this release (scanning is done on the staff portal).
External services	Lazada, GHN, Google OAuth, SMTP, AI text API	See section 1.3 Technology Declaration.

1.2 Package Diagram

[Insert Figure 2 – Package Diagram]

Figure 2. Package diagram

Package descriptions

No	Package	Description
01	be/src/modules/auth	Login, Google OAuth, JWT access / refresh tokens, two-factor authentication, password reset, guards and role decorators.
02	be/src/modules/users	Staff accounts, roles, leave status, account lock / reactivation.
03	be/src/modules/marketplace-integration	Marketplace adapters registered for Lazada and AURELLE (shared Lazada-style protocol client), OAuth connect / disconnect, encrypted shop tokens. A Tiki adapter (seller Inhouse app, dropship orders, added 31/10/2026) is also registered but is outside the Review 2 demo; the TikTok Shop adapter file is not registered.
04	be/src/modules/marketplace-webhooks	Receives and verifies AURELLE webhooks, de-duplicates events.
05	be/src/modules/marketplace-writeback	Status outbox and write-back scheduler, stock push to AURELLE.
06	be/src/modules/orders	Order import, Lazada sync scheduler, order sync settings and consolidation policy.
07	be/src/modules/order-groups	Order groups, stock reservation, staff assignment, picking, split / merge / link, express SLA and backfill schedulers.
08	be/src/modules/warehouse	Warehouses, zones, bins, SKU bin assignment, stock receipts, FIFO stock-lot ledger, inventory movements.
09	be/src/modules/categories, master-skus	Product categories, colours, internal SKUs, marketplace SKU mappings.

No	Package	Description
10	be/src/modules/product-master	Marketplace catalog sync and packaging profile of SKUs.
11	be/src/modules/packaging	Boxes, bags, filling materials, packing engine helpers, AI packing guide.
12	be/src/modules/packaging-materials	Packaging material stock, movements and rules.
13	be/src/modules/packing	Packing plans (BRKGA and CP-SAT solvers), packing job, packer assignment, packing session, settings, reports and feedback.
14	be/src/modules/shipping	Carrier quotes, live carrier service, shipping settings.
15	be/src/modules/carriers	Carrier adapters (GHN) and carrier types.
16	be/src/modules/shipments	Shipments, carrier webhook, shipment SLA scheduler, returns and marketplace returns sync.
17	be/src/modules/storefront	AURELLE back end: catalog, cart, customer auth, orders, returns, web admin, Open API mapper and webhook publisher.
18	be/src/modules/notifications	In-app notifications by role / user.
19	be/src/modules/history, activity-log	Business action log, history queries and CSV export.
20	be/src/modules/documents, mail	PDF documents (labels, picking list, receipts, manifests) and e-mail sending.
21	be/src/common	Shared constants, enums, exceptions, filters, Redis client, schema helpers and utilities.
22	fe/src (pages, components, api, lib)	Staff portal pages, UI components, typed API clients, RBAC route rules.
23	storefront/app, components, lib	Next.js pages of AURELLE, UI components, API client and developer portal helpers.
24	packer/packer	FastAPI service: models, feasibility checks, CP-SAT solving.
25	be/scripts	Seed, migration, benchmark and AURELLE mock / conformance scripts.

1.3 Technology Declaration

a. Third-party services

Service	Purpose in OptiPack AI	Data sent
Lazada Open Platform	Shop connection (OAuth) and order / catalog import.	App key and HMAC-SHA256 signature, OAuth authorization code and shop access / refresh tokens; read requests with time window, page and order IDs (orders, order items, products). Nothing is written back to Lazada.
AURELLE Open API (team-built)	Second marketplace: orders, webhooks, returns, status and stock write-back.	Signed requests with app key and shop access token: order acknowledgement; order status with tracking code, trip code, carrier and failure reason; return approve / reject decisions and exchange shipment; sellable stock per seller SKU.
GHN (Giao Hàng Nhanh) API	Shipping fee quotes, shipping orders, tracking, status webhooks, labels.	Shop ID and API token; recipient name, phone and address (province, district, ward); parcel weight and dimensions; item names, codes, quantities and prices; COD amount, insurance value, note and client order code.
Google OAuth 2.0	Staff login with Google for existing accounts.	OAuth client ID / secret, authorization code and PKCE verifier; Google returns the account e-mail and its verified flag.
Gmail SMTP	Account and password e-mails.	Recipient e-mail and name with the e-mail content: temporary password (new account),

Service	Purpose in OptiPack AI	Data sent
		password reset link, account-locked notice.
Groq API (OpenAI-compatible, model openai/gpt-oss-120b)	Wording of packing guides; template text when unavailable (timeout 20 s).	Box code, name and inner dimensions, fill ratio, packing materials and the computed placement steps; no customer, recipient or order data.
MongoDB Atlas	Managed database.	All application data over a TLS connection: staff accounts (hashed passwords), orders with recipient name, phone and address, stock, shipments, returns and the activity log.
Redis	Cache for authentication state.	Per staff account (key user:auth:<user id>, with expiry): active flag, role and must-change-password state.

b. Development technologies

Layer	Technology and version
Back end	Node.js, TypeScript 5.7, NestJS 11, Mongoose 9, @nestjs/schedule 6, @nestjs/throttler 6, Swagger 11, bcrypt 6, otplib, pdfkit, nodemailer 7, ioredis, helmet
Staff portal	React 19.2, TypeScript 6, Vite 8, React Router 7, Tailwind CSS 4, three.js 0.185, @react-three/fiber 9
AURELLE website	Next.js 15.5, React 19.1, TypeScript
Packer service	Python 3.12, FastAPI 0.142, Google OR-Tools 9.15 (CP-SAT)
Testing	Jest 30 (back end, ≥ 1,490 tests in 150 spec files on 08/11/2026), pytest (packer)
Diagrams	draw.io (architecture, use case, ERD, activity and state diagrams; exported images in docs/diagrams); docs/ERD.md lists the physical collections

c. Source code management and DevOps

- Source code on GitHub, npm workspaces monorepo (be, fe, storefront) plus packer.
- Branches: main plus one working branch per member (thi_dev, feature/thuan_be, feature/viet_ui, feature/phuong_admin, feature/phuong_ui) and short-lived topic / merge branches; changes reach main through pull requests.
- Commit messages follow Conventional Commits with the Jira ticket as scope, checked by commitlint in a Husky commit-msg hook (header ≤ 100 characters, e.g. feat(AOFP-123): …); a Husky pre-commit hook runs the back-end test suite.
- GitHub Actions ci.yml on every push and on pull requests to main: back end type check, lint and tests; staff portal type check, build and lint; AURELLE type check and build; packer ruff, mypy and pytest; Docker image builds (no push); Kubernetes manifest validation (kustomize + kubeconform).
- GitHub Actions release.yml builds and pushes the Docker images to GitHub Container Registry on push to main, on v* tags or manually; it does not deploy to a cluster.

d. Environments

Environment	Description
Local development	Each service run with npm / Python on the developer machine; database on MongoDB Atlas.
Docker Compose	docker-compose.yml with be, fe, storefront, packer, redis (and optional local mongodb, mongo-express).
Kubernetes	k8s/base with overlays local and ghcr (images from GitHub Container Registry); MongoDB stays on Atlas. Details in docs/DEPLOYMENT.md.

Environment	Description
Staging	Not available yet: no hosted staging environment is declared. Planned: deploy the Docker Compose / Kubernetes setup to a hosted server and seed demo data with the repository scripts (see Design Brief risk R1). Production is not in scope.
Deployment and rollback	Deployment is manual: release.yml only builds and pushes the images to GitHub Container Registry, tagged with the branch name, the short commit SHA (sha-<commit>), the v* version and latest; the cluster is deployed with kubectl apply -k k8s/overlays/ghcr (or k8s/overlays/local). Rollback: kubectl -n optipackai rollout undo deployment/<be | fe | storefront | packer> returns to the previous ReplicaSet, or set newTag in k8s/overlays/ghcr/kustomization.yaml to an earlier sha-<commit> tag and apply again; with Docker Compose, check out the previous Git tag or commit and run docker compose up -d --build. Data on MongoDB Atlas is not rolled back automatically; data migration scripts in be/scripts are run manually.

e. AI tools used in development

Tool	Usage
Claude (Anthropic)	Code review, implementation support, test writing, create model 3D, documentation and diagram drafting; every output is reviewed and verified against the code by the team.
Cursor	Frontend implementation support, documentation and diagram drafting, CI lock file fix, and merge-conflict resolution; every output is reviewed and verified against the code by the team.

2. Database Design

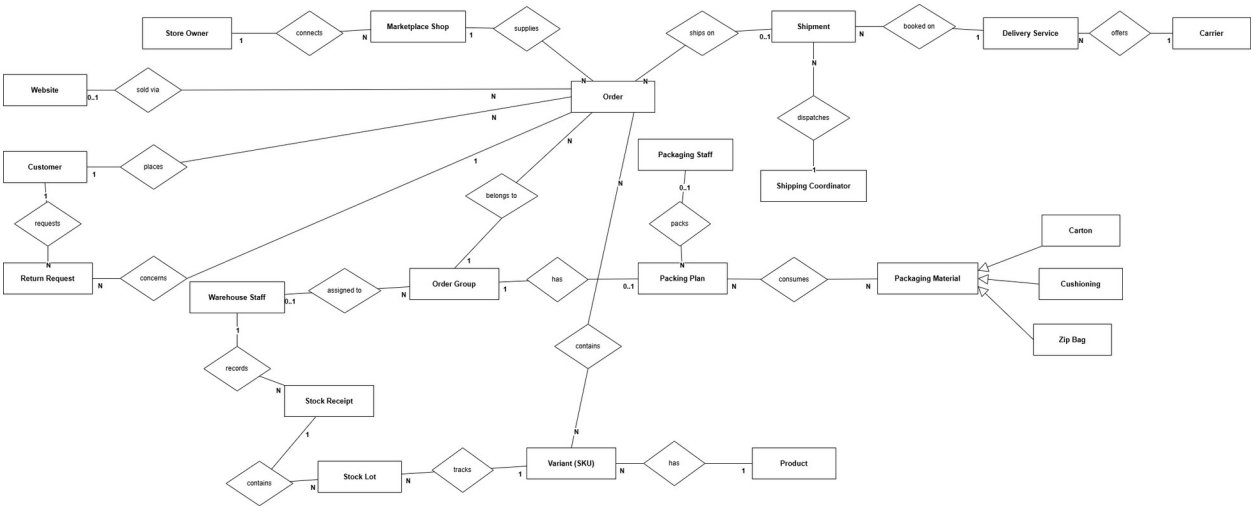


Figure 3. Logical ERD

OptiPack AI uses MongoDB. Every collection has primary key _id (ObjectId). Foreign keys are ObjectId fields that reference another collection; references inside arrays are written as array[].field. The table below lists the main collections; other collections are support data (OAuth states, counters, processed event ids, settings).

No	Collection	Description	Foreign keys (PK is _id)
01	users	Staff accounts, role, lockout, two-factor, leave.	created_by → users

No	Collection	Description	Foreign keys (PK is _id)
02	refresh_tokens / trusted_devices / login_audit_logs	Sessions, trusted devices and login audit.	user_id → users
03	marketplace_shops	Connected marketplace shops with encrypted tokens.	connected_by, disconnected_by → users
04	orders	Marketplace orders and items.	marketplace_shop → marketplace_shops; consolidated_group_id → order_groups; source_return_id → return_requests
05	order_groups	Unit of fulfilment (status, priority, flags).	assigned_staff_id → users; warehouse_id → warehouses; source_return_id → return_requests; split_from_group_id, merged_into_group_id → order_groups
06	stock_reservations	Reserved quantity per group and SKU.	order_group_id → order_groups; warehouse_id → warehouses
07	pick_events	Pick scans.	order_group_id → order_groups; warehouse_id → warehouses; bin_location_id → bin_locations; lot_allocations.lot_id → stock_lots
08	warehouses / warehouse_zones / bin_locations	Warehouse structure.	warehouse_zones.warehouse_id → warehouses; bin_locations.warehouse_id, zone_id
09	sku_bin_assignments	Stock of a SKU in a bin.	warehouse_id → warehouses; bin_location_id → bin_locations
10	stock_receipts / stock_lots	Stock receipts and FIFO lots.	stock_lots.receipt_id → stock_receipts; assignment_id → sku_bin_assignments; lines[].lot_id → stock_lots
11	inventory_movements	Stock ledger.	warehouse_id, assignment_id, bin_location_id; lot_allocations.lot_id → stock_lots
12	categories / colors / master_skus / marketplace_sku_mappings	Internal product classification, internal SKUs and seller-SKU mappings.	By code (category code, internal SKU code)
13	product_master	Marketplace catalog with packaging profile.	By platform and seller SKU
14	packing_plans	3D packing plan per group: orders, parcels, placements, scans, weighings.	order_group_id → order_groups; approved_by, rejected_by, packed_by, assigned_packer_id → users; orders[].order_id, parcels[].order_id → orders
15	packaging_materials / packaging_bags / packaging_movements / packaging_material_rules	Boxes and filling materials, their stock movements and material rules; packaging_bags holds zip bags.	packaging_movements.packing_plan_id → packing_plans; rules.updated_by → users
16	shipments / shipment_events	Shipments with carrier parcels and status history.	order_group_id → order_groups; carrier_parcels[].order_id → orders; shipment_events.shipment_id → shipments
17	return_requests	Return / exchange / failed-delivery tickets with inspection.	order_group_id, replacement_group_id → order_groups; shipment_id → shipments; source_order_id, replacement_order_id → orders; inspection[].bin_location_id

No	Collection	Description	Foreign keys (PK is _id)
18	marketplace_status_outbox	Queued status write-backs.	order_group_id, shipment_id, return_request_id, order_id
19	notifications	In-app notifications.	recipient_user_id → users; related_entity_id
20	activity_logs	Business action history.	order_group_id → order_groups; entity_id
21	customers, customer_addresses and storefront_* (products, product_variants, carts, cart_items, orders, order_items, order_events, payments, shipments, return_requests, categories, images, inventory_stocks, counters)	AURELLE website data.	Internal references of the AURELLE website

3. Main Business Workflows

The system has six main business workflows. Each workflow involves at least two actors and changes the state of a main entity. Each workflow is described by one activity diagram (swimlane per actor), and two overview diagrams show how the workflows connect. The diagrams are drawn with draw.io; the exported images are kept in the repository at docs/diagrams/activity/drawio.

ID	Workflow	Actors	Trigger	Outcome / main entity state	Owner
WF-01	Order Intake & Consolidation	Customer, Marketplace, OptiPack System, Store Owner / Coordinator, Warehouse Staff	Customer places an order	Order Group: awaiting_packaging → picking	Thuận
WF-02	Picking	Warehouse Staff, OptiPack System, Packaging Staff	Group in picking with an assigned picker	Order Group: picking → picked, or → partial_needs_review → picked / picking	Phượng
WF-03	Packing	OptiPack System, Packaging Staff, Reviewer	Group reaches picked	Packing Plan: computing → ready → approved → packing → packed (or rejected, failed, superseded); Order Group: picked → pending_approval → approved_for_packing → packed	Việt
WF-04	Shipping (GHN and Express Self-Delivery)	Shipping Coordinator, OptiPack System, GHN, Customer, Warehouse Staff	Coordinator creates a shipment	Shipment: awaiting_pickup → in_transit → out_for_delivery → delivered / delivery_failed → returning_to_warehouse → returned_to_warehouse (express self-delivery starts at out_for_delivery); a carrier order cancelled before pickup ends in cancelled	Thuận
WF-05	Return / Exchange via AURELLE	Customer, AURELLE Website, OptiPack System, Store Owner,	Customer submits a return /	Return Request: requested → awaiting_receipt → received → closed (rejected, canceled)	Thi

ID	Workflow	Actors	Trigger	Outcome / main entity state	Owner
		Warehouse Staff	exchange request		
WF-06	Stock Receiving by Lot	Warehouse Staff, OptiPack System, AURELLE Website	Goods arrive from supplier	Stock Lot created; stock-shortage flag of groups cleared	Thi

3.1 End-to-End Overview

The end-to-end view is split into two flows. The customer purchase flow follows one order from the marketplace through WF-01 to WF-05; the supplier stock receiving flow shows how new goods enter the warehouse and is detailed in WF-06.

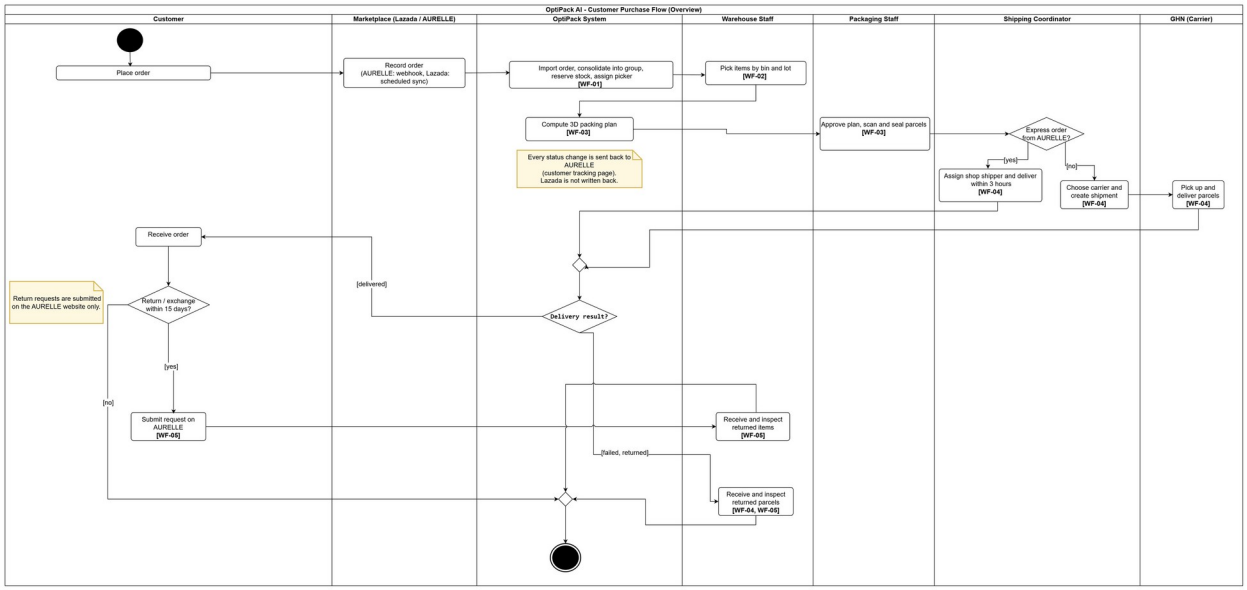


Figure 4. Customer purchase flow (overview of WF-01 to WF-05)

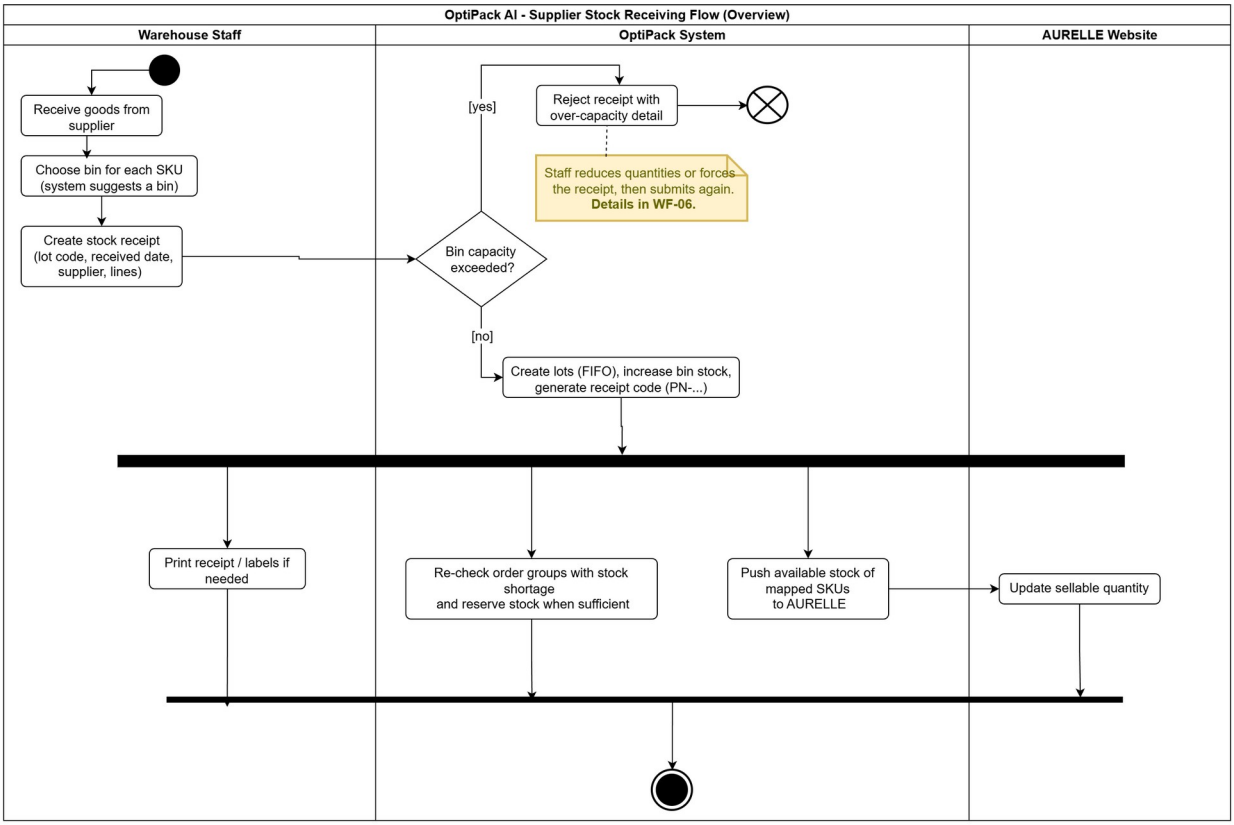


Figure 5. Supplier stock receiving flow (overview of WF-06)

3.2 WF-01 Order Intake & Consolidation

An order placed on Lazada or AURELLE is imported (AURELLE order-changed webhook, or scheduled sync every 5 minutes by default) and saved. A cancelled or no longer fulfillable order only re-checks its group. Otherwise the order joins an open group with the same consolidation key or starts a new group; express AURELLE orders always start their own group. A new group gets the least-busy Warehouse Staff, stock is reserved and the group moves to picking. Recipient-name mismatch and stock shortage are notified, and the Store Owner / Coordinator can split or merge groups before any item is scanned.

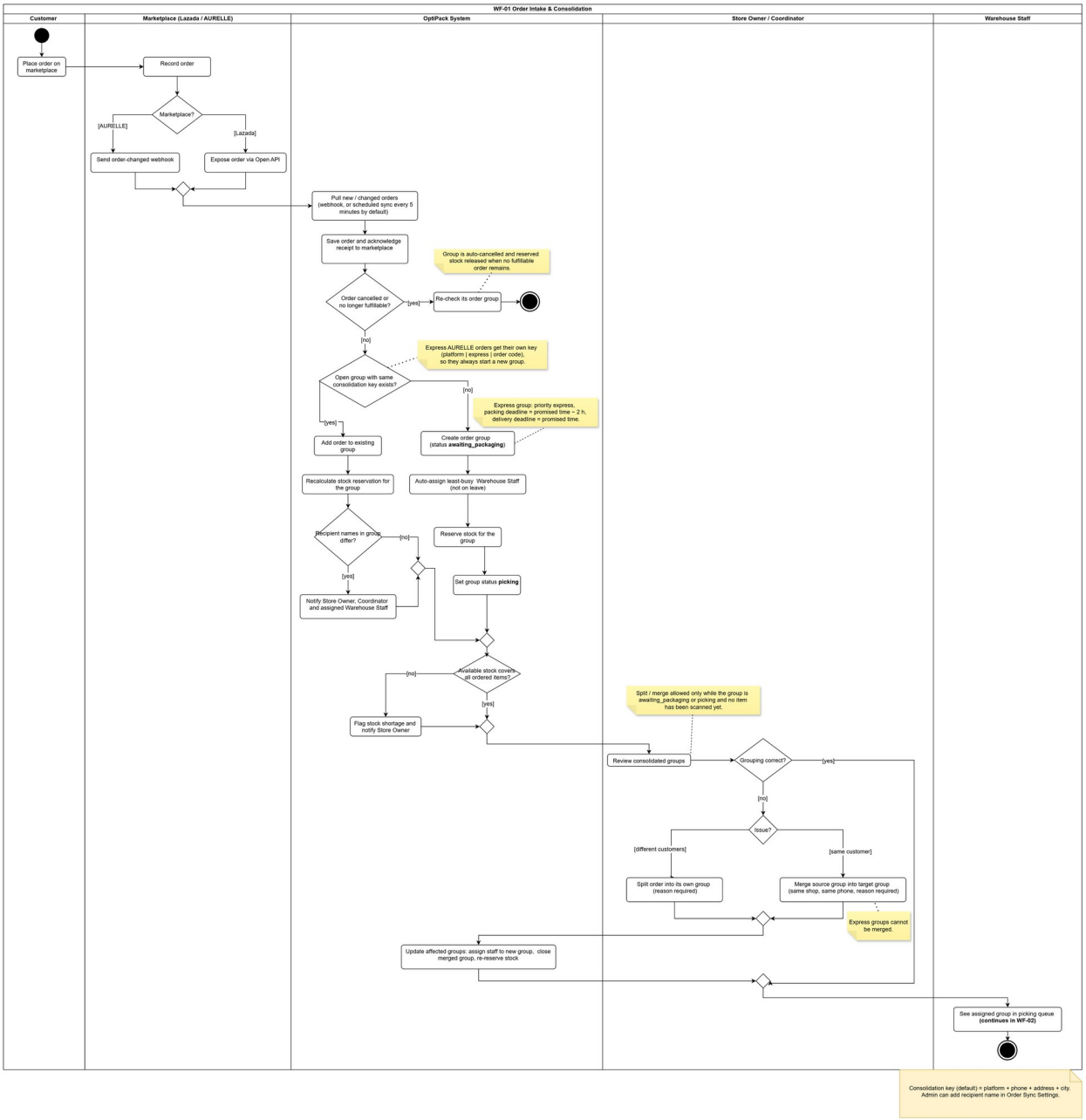


Figure 6. Activity diagram – WF-01 Order Intake & Consolidation

3.3 WF-02 Picking

The assigned Warehouse Staff scans items bin by bin; the system checks each scan and deducts stock from the oldest lot (FIFO). The first valid scan tells AURELLE customers that the warehouse is preparing the order. When all SKUs are picked the group becomes picked and unused reservations are released. When items are missing the group goes to partial_needs_review and the Packaging Staff either ships the available items (the shortage is allocated to the oldest order first and customers are notified) or starts a new pick round. A picked group is queued for packing-plan computation (WF-03).

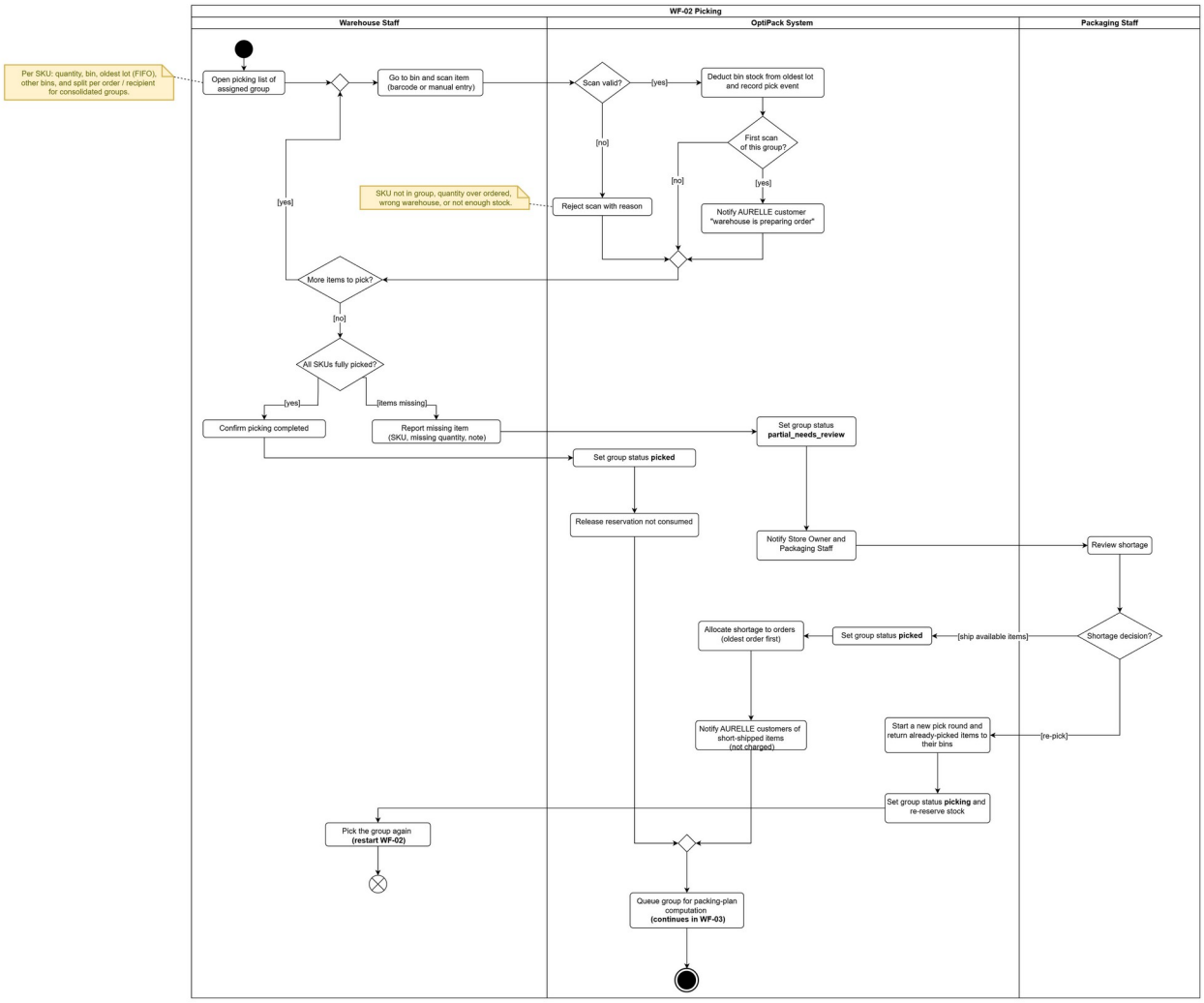


Figure 7. Activity diagram – WF-02 Picking

3.4 WF-03 Packing

A background job computes a 3D packing plan for each order of a picked group. If a SKU packaging profile is not confirmed, the group is blocked until the SKU is measured; otherwise the plan becomes ready, the group goes to pending_approval and a packer is assigned. The Packaging Staff approves or rejects the plan; a rejection sends the group back to picking, to a manual plan approved by another person, or to a recomputed plan. While packing, every scan is checked against the parcel, each parcel is sealed and weighed, and a parcel outside the weight tolerance is held for a Reviewer who is not the person who sealed it. After the last parcel the plan and the group become packed and AURELLE customers are notified.

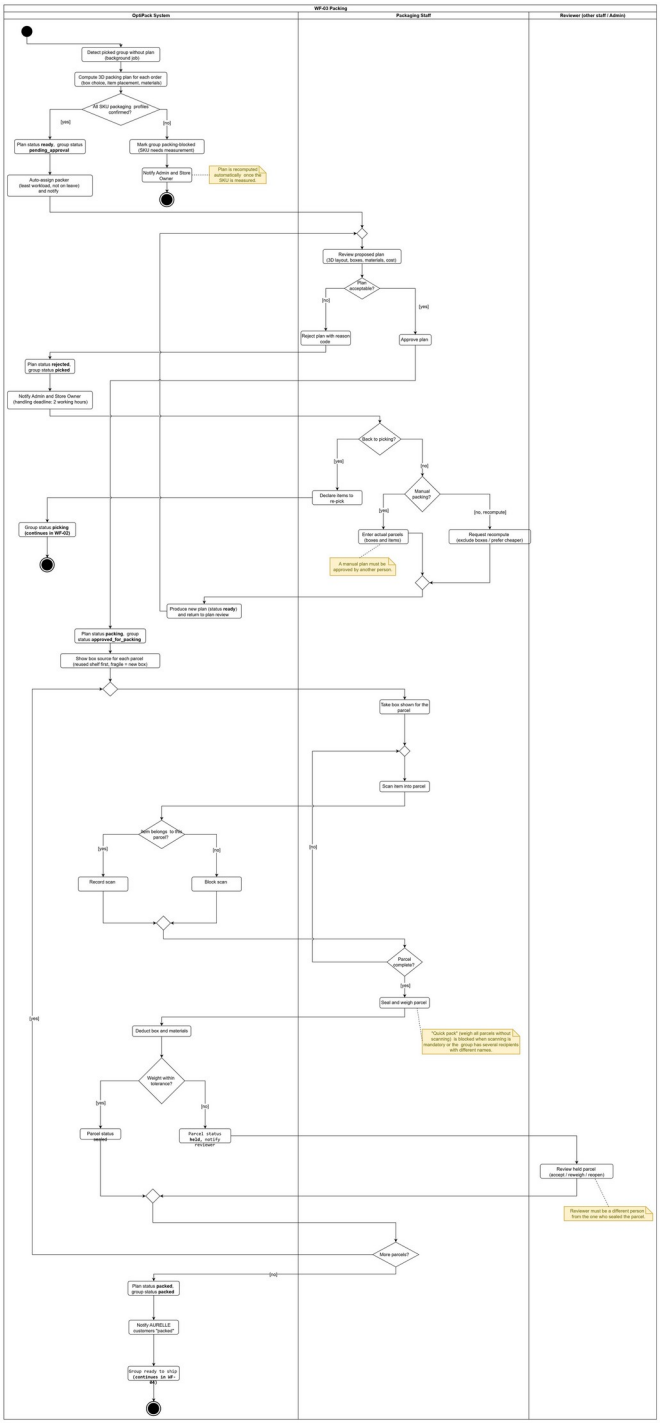


Figure 8. Activity diagram – WF-03 Packing

3.5 WF-04 Shipping (GHN and Express Self-Delivery)

The Shipping Coordinator opens packed groups. An express AURELLE group is delivered by the shop itself: the Coordinator chooses a shop shipper, the shipment starts at out_for_delivery with the promised time as due time, and the shipper's name and phone are sent to AURELLE. Other groups go by carrier: the Coordinator compares quotes and creates the shipment (awaiting_pickup), one GHN order is

booked per parcel, and parcels rejected by GHN are re-booked after the delivery address is corrected. GHN status updates (webhook or sync) move the shipment through in_transit and out_for_delivery to delivered (a COD order is marked paid) or to delivery_failed. If the parcel is not redelivered it returns to the warehouse; scanning it there sets the group to returned and creates a failed-delivery return ticket (WF-05). Each status change is sent to AURELLE customers.

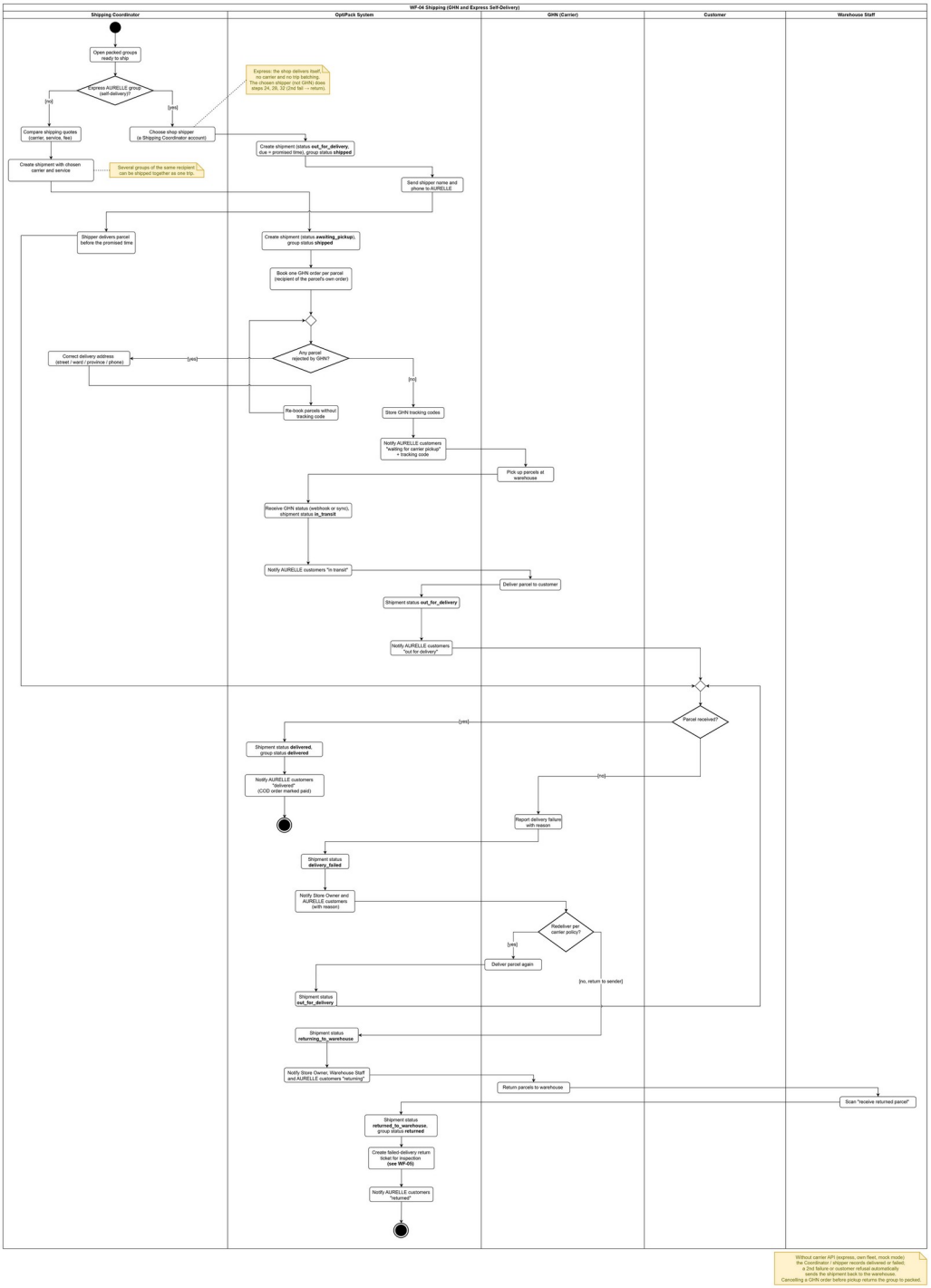


Figure 9. Activity diagram – WF-04 Shipping (GHN and Express Self-Delivery)

3.6 WF-05 Return / Exchange via AURELLE

A customer submits a return, refund-only or exchange request on AURELLE. The website refuses it when the order is not delivered, is older than 15 days, already has an open request, or the exchange variant is out of stock. OptiPack pulls the request, creates a return ticket and notifies the Store Owner, who approves or rejects it with a reason. A refund-only ticket is closed at once; otherwise the customer sends the goods back, the Warehouse Staff receives and inspects each SKU (restock / quarantine / discard), the ticket is closed and groups waiting for stock are re-checked. Each step is sent to AURELLE, which records the refund; for an exchange a replacement order EXC-… (price 0) and a new order group are created and go through WF-02 to WF-04.

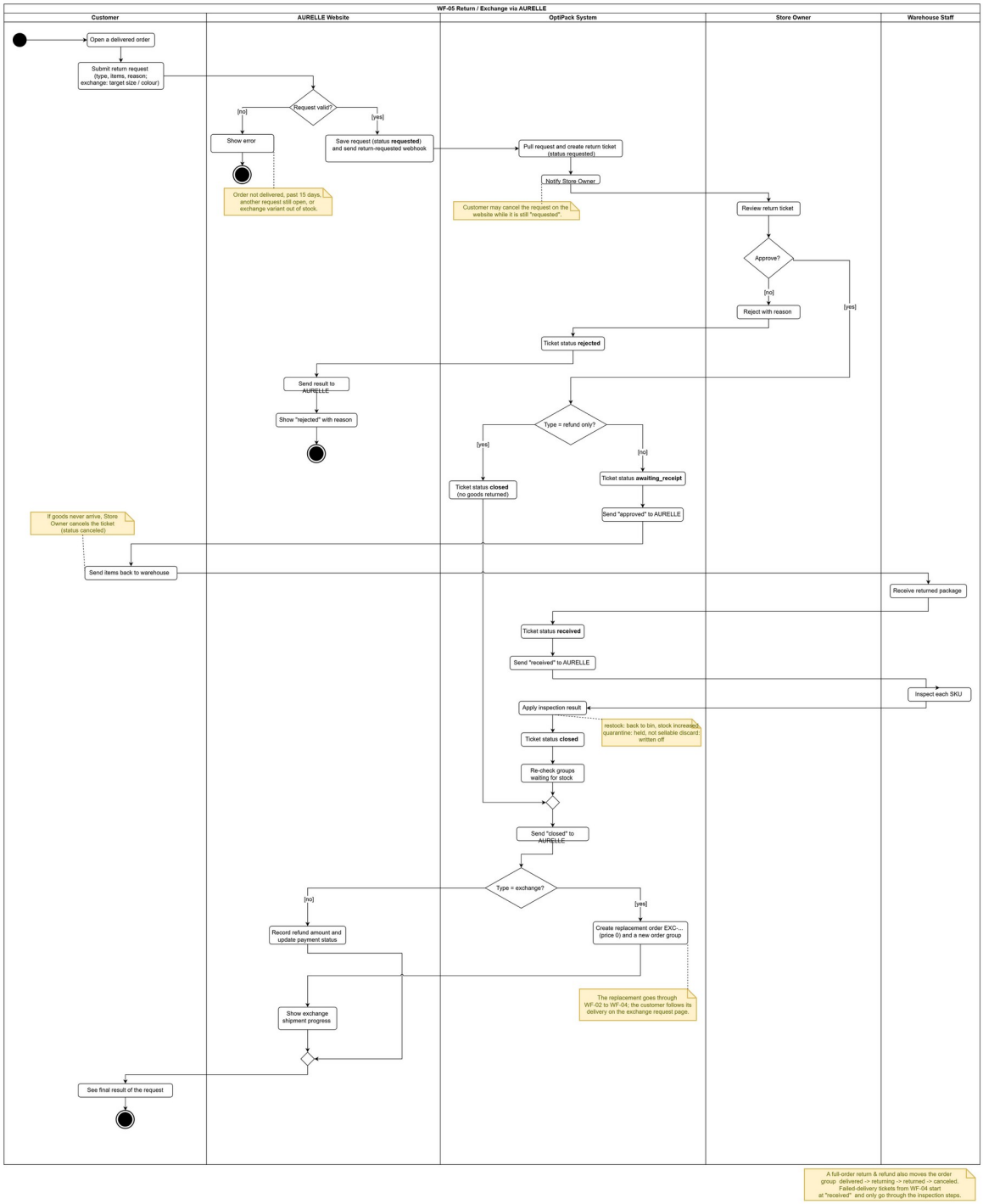


Figure 10. Activity diagram – WF-05 Return / Exchange via AURELLE

3.7 WF-06 Stock Receiving by Lot

Warehouse Staff chooses a bin for each SKU line (the system suggests one). If the SKU is not yet assigned to that bin the system creates the assignment, asking for confirmation first when the SKU is already in another bin of the warehouse. A stock receipt is then created; if a bin capacity would be exceeded the receipt is rejected, and staff can reduce quantities or force it. Otherwise each line becomes a FIFO lot,

bin stock and the inventory ledger are updated and a receipt code PN-… is generated. In parallel, groups with stock shortage are re-checked and reserved when stock is now sufficient, and available stock of mapped SKUs is pushed to AURELLE.

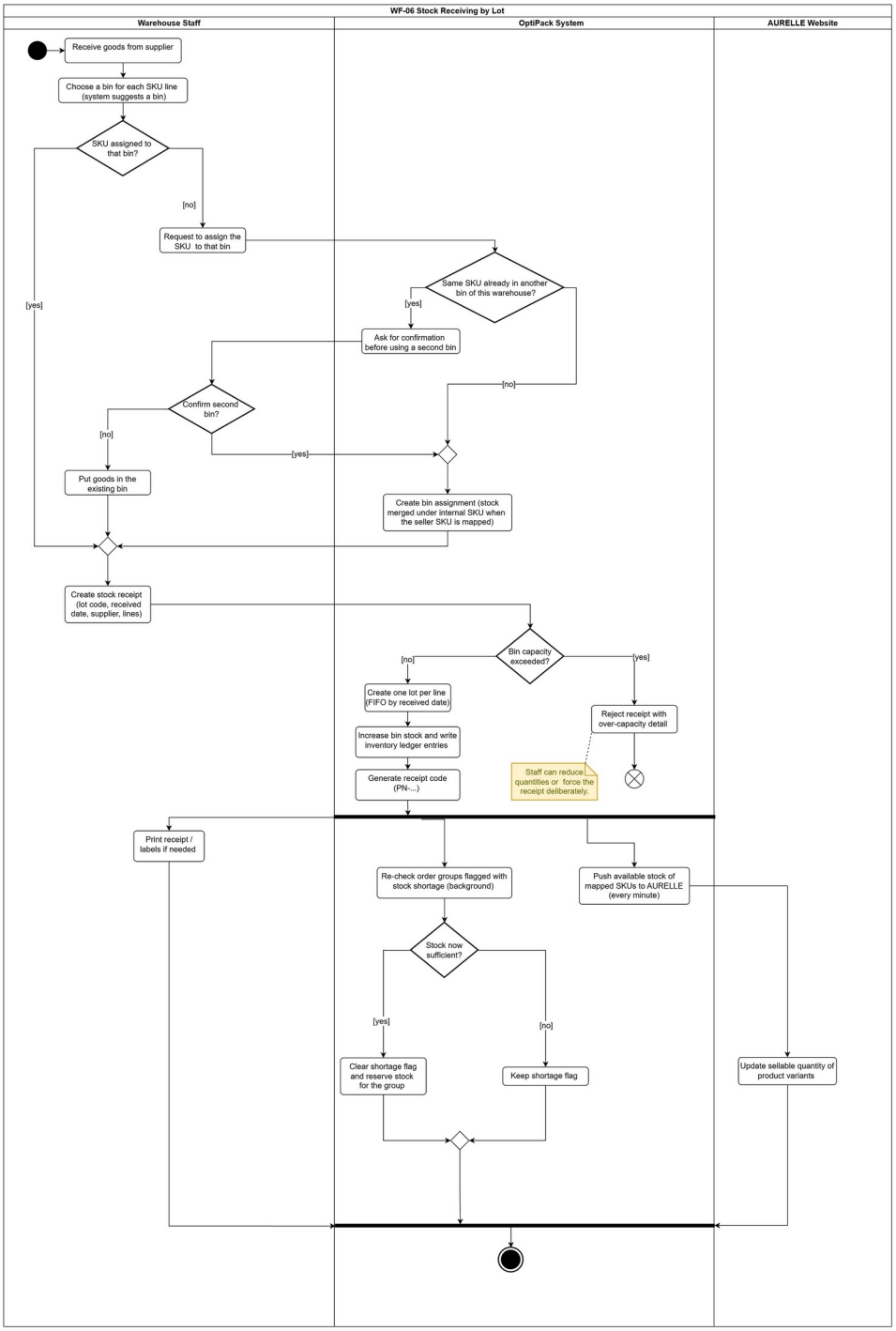


Figure 11. Activity diagram – WF-06 Stock Receiving by Lot

3.8 State Machine Diagrams

State machine diagrams of the main entities whose states are changed by the workflows above.

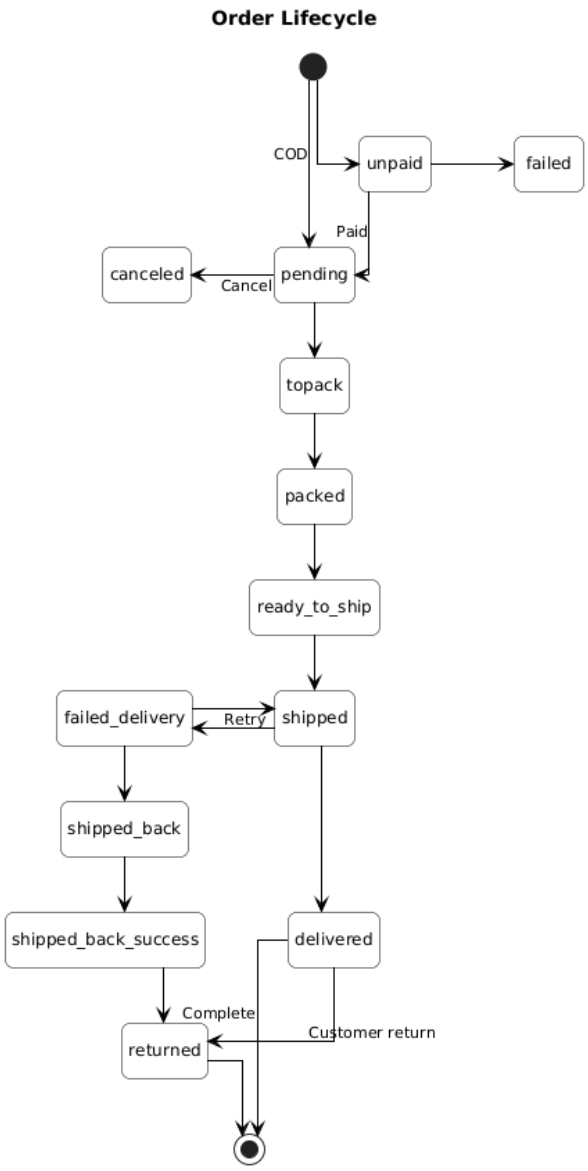


Figure 12. State machine diagram – Order (marketplace order status)

The Order status mirrors the status of the order on the marketplace (Lazada status names; AURELLE uses the same names) and is updated by order sync and webhooks; OptiPack AI tracks its own fulfilment progress on the Order Group (states listed in the workflow table above).

4. Detailed Design

For each feature, the class diagram shows the controller, services, schemas and external adapters involved, and the sequence diagrams show the main interactions of the functions listed in Report 3.

4.1 Authentication and Account Management

4.1.1 Class Diagram

Main classes: AuthController, AuthService, TokenService, MfaService, JwtStrategy, RolesGuard, UsersController, UsersService, MailService.

[Insert figure – Class Diagram: Authentication and Account Management]

4.1.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Log in (with two-factor)]

[Insert figure – Sequence Diagram: Reset password]

[Insert figure – Sequence Diagram: Create user account]

4.2 Marketplace Integration and Order Intake

4.2.1 Class Diagram

Main classes: MarketplaceIntegrationController / Service, LazadaAdapter, AurelleAdapter, LazadaOrderSyncScheduler, MarketplaceWebhooksService, OrdersService, OrderGroupsService, StockReservationService, StaffAssignmentService.

[Insert figure – Class Diagram: Marketplace Integration and Order Intake]

4.2.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Connect marketplace shop]

[Insert figure – Sequence Diagram: Synchronize and consolidate orders]

[Insert figure – Sequence Diagram: Receive AURELLE webhook]

4.3 Warehouse and Inventory

4.3.1 Class Diagram

Main classes: WarehouseController, WarehouseService, StockLotLedger, MasterSkusService, CategoriesService, ProductMasterService, DocumentsService.

[Insert figure – Class Diagram: Warehouse and Inventory]

4.3.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Assign SKU to bin]

[Insert figure – Sequence Diagram: Create stock receipt]

4.4 Picking

4.4.1 Class Diagram

Main classes: OrderGroupsController, OrderGroupsService, StockReservationService, NotificationsService.

[Insert figure – Class Diagram: Picking]

4.4.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Pick item]

[Insert figure – Sequence Diagram: Report missing items and decide partial]

4.5 Packing

4.5.1 Class Diagram

Main classes: PackingPlanController, PackingPlanService, PackingJobService, PackerAssignmentService, PackingSessionService, PackingSettingsService, BRKGA / CP-SAT solver, PackingGuideAiService.

[Insert figure – Class Diagram: Packing]

4.5.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Compute packing plan]

[Insert figure – Sequence Diagram: Approve / reject plan]

[Insert figure – Sequence Diagram: Seal and weigh parcel]

4.6 Shipping

4.6.1 Class Diagram

Main classes: ShippingController, ShippingService, LiveCarrierService, GhnAdapter, ShipmentsController, ShipmentsService, CarrierWebhookController.

[Insert figure – Class Diagram: Shipping]

4.6.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Create shipment]

[Insert figure – Sequence Diagram: Process carrier status update]

4.7 Returns and Exchanges

4.7.1 Class Diagram

Main classes: ReturnsController, ReturnsService, ReturnsMarketplaceSyncService, StorefrontReturnsService.

[Insert figure – Class Diagram: Returns and Exchanges]

4.7.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Approve / reject return]

[Insert figure – Sequence Diagram: Receive and inspect returned items]

4.8 Marketplace Write-back

4.8.1 Class Diagram

Main classes: MarketplaceWritebackService, MarketplaceWritebackScheduler, MarketplaceStockPushService.

[Insert figure – Class Diagram: Marketplace Write-back]

4.8.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Write status back to AURELLE]

[Insert figure – Sequence Diagram: Push stock to AURELLE]

4.9 AURELLE Website

4.9.1 Class Diagram

Main classes: StorefrontController, CustomerAuthService, StorefrontCartService, StorefrontOrdersService, StorefrontAdminService, AurelleWebhookPublisher.

[Insert figure – Class Diagram: AURELLE Website]

4.9.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: Place order]

[Insert figure – Sequence Diagram: Cancel order]

[Insert figure – Sequence Diagram: Request return / exchange]

4.10 History and Notifications

4.10.1 Class Diagram

Main classes: HistoryService, NotificationsService.

[Insert figure – Class Diagram: History and Notifications]

4.10.2 Sequence Diagram(s)

[Insert figure – Sequence Diagram: View history / export CSV]